

UFMFRR-15-M

MACHINE VISION

GROUP REPORT

Machine Vision Project Group Report

Authors :

Nathaphong Chinyooyong (25061794)

Sangyoon Lee (25061816)

Sizhe Wu (25068989)

Ben Fortson (25061867)

Jamie Passfield (22035735)

Instructor:

Dr Wenhao Zhang

Dr Disi Chen

Gytis Bernotas



December 16, 2025

No.	Name of group members	Contribution to project	Contribution to report	Signature
1	Nathaphong Chinyooyong	Implemeted an image-processing algorithm together with the machine learning model, texture extraction, image-processing, Augmentation code, ground cutting algorithm, overlap window cutting algorithm	Section2.1(30%) Section2.2(60%) Section2.3(60%)	Nathaphong
2	Sangyoon Lee	Preparing the dataset, validation code, clahe, gamma- correction, extract round feature of the apple	Section1(100%) Section3(100%)	Sangyoon Lee
3	Sizhe Wu	Watershed algorithm, hsv color segmentation, image label, debugging tool	Learning logs group project development(100%)	Sizhe Wu
4	Ben Fortson	Try K-mean algorithm, post-processing NMS	Learning logs tutorial integration(100%)	Ben Fortson
5	Jamie Passfield	Try the super pixel algorithm and grabcut, image label, debugging tool	Section2.1(70%) Section2.2(40%) Section2.3(40%)	JPassfield

Contents

Contents	ii
List of Figures	iii
1. Introduction	1
1.1 Course Progression	1
1.2 Technical Background	1
1.3 Project Objective	1
2. Methodology	2
2.1 Dataset Preparation and Machine-Learning Development	2
2.1.1 Lighting Conditions	2
2.1.2 Color Similarity	3
2.1.3 Apple Overlap	3
2.1.4 Occlusion	3
2.1.5 Ground Clearing	3
2.1.6 Dataset Utilization	4
2.2 Image Processing	4
2.2.1 Illumination Normalization	5
2.2.2 Texture-Based Background Suppression	5
2.2.3 Morphological Refinement and Contour Filtering	5
2.2.4 Color-Based Segmentation in HSV Space	6
2.2.5 Region of Interest Extraction and Spatial Partitioning	6
2.2.6 Post-Detection Merging for Counting Stability	6
2.2.7 Contribution to Apple Counting	7
2.3 Machine Learning	7
2.3.1 Model Selection: Bounding Box vs. Segmentation	7
2.3.2 Training Strategy and Dataset Partitioning	8
2.3.3 Model Capacity and Architecture Choice	8
2.3.4 Addressing Project-Specific Challenges	9
3. Results and Evaluation	9
3.1 Preprocessing & Pipeline Optimization	9
3.2. Performance Analysis of Version 5	11
3.3. Evolution of Models	12
References	14

List of Figures

Fig. 1. Final apple counting algorithm pipeline.	2
Fig. 2. Visualization of the preprocessing pipeline.	10
Fig. 3. Comparison of detection results before and after applying NMS.	10

List of Tables

TABLE I MODEL PERFORMANCE COMPARISON	12
--	----

1. Introduction

1.1 Course Progression

This project bridges classical image processing and advanced machine learning, demonstrating how low-level pixel manipulation can serve as a critical pre-processing step to optimize deep learning-based object detection in apple orchard environments.

1.2 Technical Background

Apple detection faces challenges like variable illumination and occlusion. While classical methods lack robustness against environmental changes [1], and two-stage detectors (e.g., Faster R-CNN) are computationally intensive [2], one-stage detectors like YOLO prioritize real-time inference [3], but often struggle with small, clustered objects due to information loss [4]. Rather than relying solely on computationally expensive instance splitting, this project adopts a Data-Centric AI approach [5]. An investigation was conducted on a hybrid pipeline that utilizes classical image processing (e.g., ROI cropping, tiling, filtering) to enhance feature visibility before feeding data into a YOLO detector [6].

1.3 Project Objective

The primary objective is to develop a robust apple counting algorithm by integrating image processing with machine learning. While deep learning models are powerful, their efficacy depends on data quality. This project empirically demonstrates that improving input quality through conventional pre-processing (such as segmentation and texture detection) significantly enhances the accuracy and resilience of the model compared to using raw, unprocessed images.

Default: Model training executed using raw, unprocessed images.

Enhanced: Model training executed using images pre-processed with techniques such as segmentation, morphological methods, and texture detection to accentuate apple

features and mitigate background interference.

2. Methodology

2.1 Dataset Preparation and Machine-Learning Development

The scenario requires the algorithm to detect the apples within the orchard however, apple detection in orchards face environmental challenges, categorized into four main aspects:

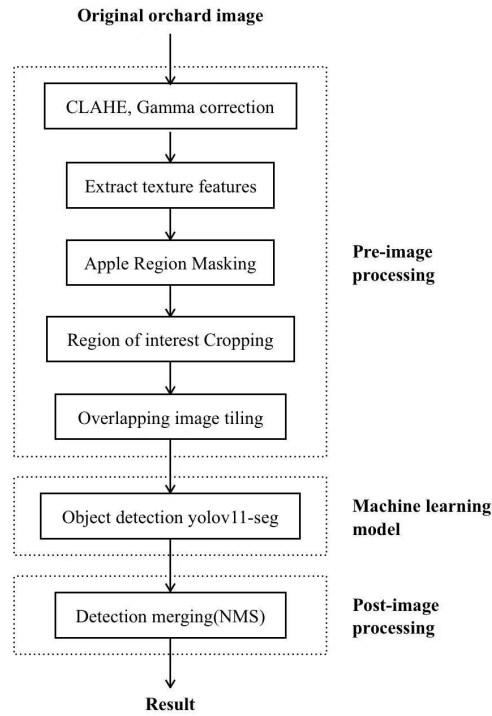


Fig. 1. Final apple counting algorithm pipeline.

2.1.1 Lighting Conditions

Lighting conditions in orchards can vary due to sunlight levels and direction, shadows from leaves, and weather conditions. **Contrast-Level Adaptive Histogram Equalization (CLAHE)** is used within preprocessing to enhance local contrast whilst limiting noise amplification, reducing the effects of uneven illumination. Whilst it improves overall image clarity, it does not fully illuminate low-visibility regions.

2.1.2 Color Similarity

Color similarity between apples and foliage can cause the algorithm to identify multiple close apples as one or leaves as apples, vice versa. The leaves' hard edges are excluded to focus on the smooth apple surface, so grayscale thresholding and morphological operations are applied to reduce the resulting noise. After color filtering a clean mask passes a color segmentation state to reduce background interference.

2.1.3 Apple Overlap

Overlapping apples can cause boundaries to merge and be misrepresented as one apple due to factors such as proximity and colors, exacerbated by image blur from camera distance. Machine learning was implemented to learn the visual patterns and instance-level separation from annotated training data.

2.1.4 Occlusion

Apples are generally represented as circular and possess distinguishable color and texture characteristics. Shape based detection alone is unreliable, due to shape occlusion from foliage or light. HSV color segmentation is used to remove large fragments of background. Leaves and branches contain strong edges, so edge-based filters can suppress more background features for better apple detection.

2.1.5 Ground Clearing

Fallen apples are not to be counted, so texture and contextual information are considered to ensure this. Ground regions often exhibit sharp textures and color distributions similar to soil. Analyzing texture sharpness and color patterns around apples allows for grounded apples to be excluded.

2.1.6 Dataset Utilization

The apple detection system was created with the MinneApple dataset [7] and I61I's dataset [8] used throughout the development of five successive model versions. Version 1 initially used the MinneApple dataset, as it provides annotated orchard images with instance-level segmentation levels. The dataset offered a strong baseline for learning apple appearance under real orchard conditions and enabled initial evaluations of segmentation-based detection performance. Version 2 additionally utilizes I61I's dataset, with the assumptions that close-up camera views in a stable environment yielded more reliable counting results, by focusing on near field apples. The model displayed improved counting accuracy, however a bias of only detecting larger apples was induced. Version 3 addressed previous limitations, as it was observed that models trained primarily on close apples struggled with distant and occluded apples found in dense orchard scenes. To mitigate this shortcoming, a **region of interest (ROI)** strategy where apples were extracted from regions from the image preprocessing was introduced. Version 4 added further refinement by addressing dataset variability in image resolution, apple and orchard structure. Subdividing larger images into tiles allows for the model to focus on consistent apple-scale regions, to reduce background clutter confusion. Version 5 transitions from bounding boxes to segmentation-based detection, which provides more precise apple boundaries than bounding boxes and is better suited for overlapping and clustered apples. The dataset was refined to prioritize high-resolution images with clear instance-level segmentation, sourced from MinneApple dataset, while increasing the proportion of close-to-camera apples to improve instance separation and counting reliability. This dataset remains suitable for orchard conditions by retaining variations in illumination, background clutter, and partial occlusion.

2.2 Image Processing

The image processing stage reduces environmental variability and suppresses

non-apple regions before applying learning-based detection, improving instance separability and apple counting accuracy in complex orchard scenes. Additionally, image processing is used as a structured filtering and localization mechanism that constrains the search space for the detection model.

2.2.1 Illumination Normalization

Orchard images exhibit strong illumination variability, due to shadows, sunlight and canopy occlusion. Therefore, CLAHE is applied to the luminance channel of the image, enhancing local contrast, while preventing excessive noise amplification in homogenous regions. Instead of global histogram equalization, CLAHE is preferred due to its spatial adaptiveness and suitability for uneven illumination. Gamma correction then adjusts overall brightness distribution and stabilizes intensity variations across images. These steps improve visual consistency and ensure apple color and texture features remain distinguishable under varied lighting, benefiting subsequent segmentation and detection stages.

2.2.2 Texture-Based Background Suppression

Apples generally have smooth surface textures, whereas orchard backgrounds, like leaves, branches and soil, contain strong edges and high-frequency texture patterns. Edge-dominant regions can be suppressed to reduce background interference, although aggressive texture suppression introduces pixel-level noise and fragmented regions. **Grayscale thresholding** converts the image into binary representation, isolating potential apple regions and discarding low-confidence background pixels. Empirically selected threshold values balance sensitivity and noise suppression across light conditions.

2.2.3 Morphological Refinement and Contour Filtering

Binary masks produced after thresholding often contain small, isolated regions caused by residual noise. **Morphological opening**, consisting of erosions then dilation

removes the small artifacts, while preserving larger apple-sized structures. Structuring element size is based upon expected minimum apple size in image, ensuring apples are not removed. Afterwards, **contour-area filtering** retains only connected components whose area falls within a predefined range. This range is determined using dataset statistics and excludes regions that are either too small(noise), or too large (background structures, such as the sky or tree trunks). This significantly reduces false positives and improves the precision of apple-like region selection.

2.2.4 Color-Based Segmentation in HSV Space

After texture and morphological filtering, **HSV color segmentation** is applied to refine apple region candidates. HSV color space is robust against illumination change, as hue and saturation components are less sensitive to lighting intensity than RGB values. Multiple defined hue ranges account for different apple appearances including green, red and transitional ripening apples. These ranges are derived from the MinneApple dataset segmentation mask analysis and validated empirically. While this substantially suppresses foliage and background regions, recall over precision is favored to avoid prematurely discarding partially occluded apples.

2.2.5 Region of Interest Extraction and Spatial Partitioning

The refined apple masks extract ROIs from the original image, removing large portions of irrelevant background and reducing computational overhead. To handle scale variation and densely clustered apples, each ROI is divided into overlapping fixed resolution tiles. This ensures apples remain within consistent scale range and prevents larger apples from dominating feature learning. **Overlapping tiles** assist in avoiding missing apples near tile boundaries and improve robustness in highly congested regions.

2.2.6 Post-Detection Merging for Counting Stability

Since overlapping tiles may detect the same apple multiple times, during

post-processing **Non-Maximum Suppression (NMS)** merges duplicate detections based on spatial overlap and confidence scores. This is critical for apple counting, as duplicate detection causes over-counting errors. By retaining the most confident detection per instance, NMS ensures a one-to-one correspondence between detected instances and physical apples.

2.2.7 Contribution to Apple Counting

Overall, the image processing pipeline reduces background complexity, normalizes illumination, and constrains detection to apple-like regions. These design choices improve instance-level separability, reduce false positives caused by background clutter and fallen apples, and enhance recall in occluding and overlapping scenarios. By shifting complexity away from the detection model and into structured preprocessing, the pipeline improves counting accuracy and stability under real orchard conditions.

2.3 Machine Learning

This project adopts a supervised learning approach for apple detection and counting, using deep learning–based object detection models trained on annotated orchard images. The dataset is partitioned into **training, validation, and test sets** to support model learning, hyperparameter tuning, and unbiased performance evaluation. Normally, a **70% / 15% / 15% split** is used, providing a balanced trade-off between learning capacity and evaluation reliability for the available dataset size.

2.3.1 Model Selection: Bounding Box vs. Segmentation

Two model paradigms were investigated: **bounding-box–based detection** and **instance segmentation–based detection**, both implemented using YOLO architectures. Bounding-box detection models are computationally efficient and stable for localizing regions of interest, particularly when object boundaries are ambiguous. However, in orchard environments, apples frequently overlap or cluster together,

limiting the effectiveness of bounding boxes for accurate counting, as multiple apples may be enclosed within a single box or duplicated across predictions.

Instance segmentation models, by contrast, predict pixel-level masks for each detected object. This representation is suited for separating overlapping apples and handling partial occlusion, which is common in dense orchard scenes. Consequently, segmentation-based models were favored for the final system. A known drawback of segmentation in outdoor data has increased sensitivity to background noise and annotation imperfections. This issue is mitigated in this project through extensive image preprocessing and region-of-interest filtering prior to inference.

2.3.2 Training Strategy and Dataset Partitioning

The training set is for learning model parameters, while the validation set is for hyperparameter tuning, confidence threshold selection, and early stopping decisions. The test set is reserved for final performance evaluation. Although a 70/15/15 split is commonly adopted, the split ratio is adjusted depending on dataset size and model capacity. When the dataset is relatively small or the model capacity is limited, an excessively large validation set can reduce the effective training data and hinder generalization. Therefore, alternative splits such as **85% / 10% / 5%** are considered to prioritize feature learning while retaining sufficient validation data for model selection. This reflects an intentional trade-off between training diversity and evaluation stability rather than a fixed rule.

2.3.3 Model Capacity and Architecture Choice

Model size is selected based on computational constraints and feature complexity. Smaller models offer faster inference but may lack sufficient representational capacity to capture the visual diversity of apples under varying lighting, scale, and occlusion. Larger models provide stronger feature extraction but incur significantly higher computational cost and training time. In this project, a **YOLO medium-sized architecture** is chosen as a compromise between accuracy and efficiency. This model

provides adequate capacity to learn complex orchard features while remaining practical for iterative experimentation and deployment.

2.3.4 Addressing Project-Specific Challenges

The selected segmentation-based YOLO model addresses the key challenges identified in this project. Pixel-level masks improve instance separation for overlapping apples, while deep feature learning enables robustness to color similarity and partial occlusion. Combined with the preprocessing pipeline, the model focuses on apple-like regions and suppresses background noise, improving both detection precision and counting reliability. The integration of model architecture, training strategy, and dataset design ensures that the learning-based component is closely aligned with the practical requirements of apple counting in real orchard environments.

3. Results and Evaluation

The hybrid pipeline is evaluated on Close-range and Far-range datasets, with a primary focus on the efficacy of preprocessing and the performance of the final model (Version 5).

3.1 Preprocessing & Pipeline Optimization

To improve training and inference, a three-stage optimization pipeline was implemented.

1. **Dynamic Color Masking:** The `click_hsv_dynamic` function was utilized to dynamically extract HSV thresholds for red and green apple regions from test images. This generated a primary Mask (ROI) that filters out backgrounds, compensating for the sensitivity of traditional HSV methods to lighting changes.
2. **Texture Extraction:** Simple color filtering often misclassified leaves or ground weeds as apples. To address this, `texture_extraction` was introduced to highlight

surface texture features. This created an additional mask separating smooth apple surfaces from rough background noise, refining the model's focus area.

3. **Grid Tiling & NMS:** In the MinneApple dataset (Far-range), apples appear extremely small, leading to detection failures. A tiling technique was applied to divide the image into a grid of sections. To prevent duplicate detections at the boundaries, an overlap area was defined, and the Non-Maximum Suppression (NMS) algorithm was utilized to effectively remove redundant bounding boxes.

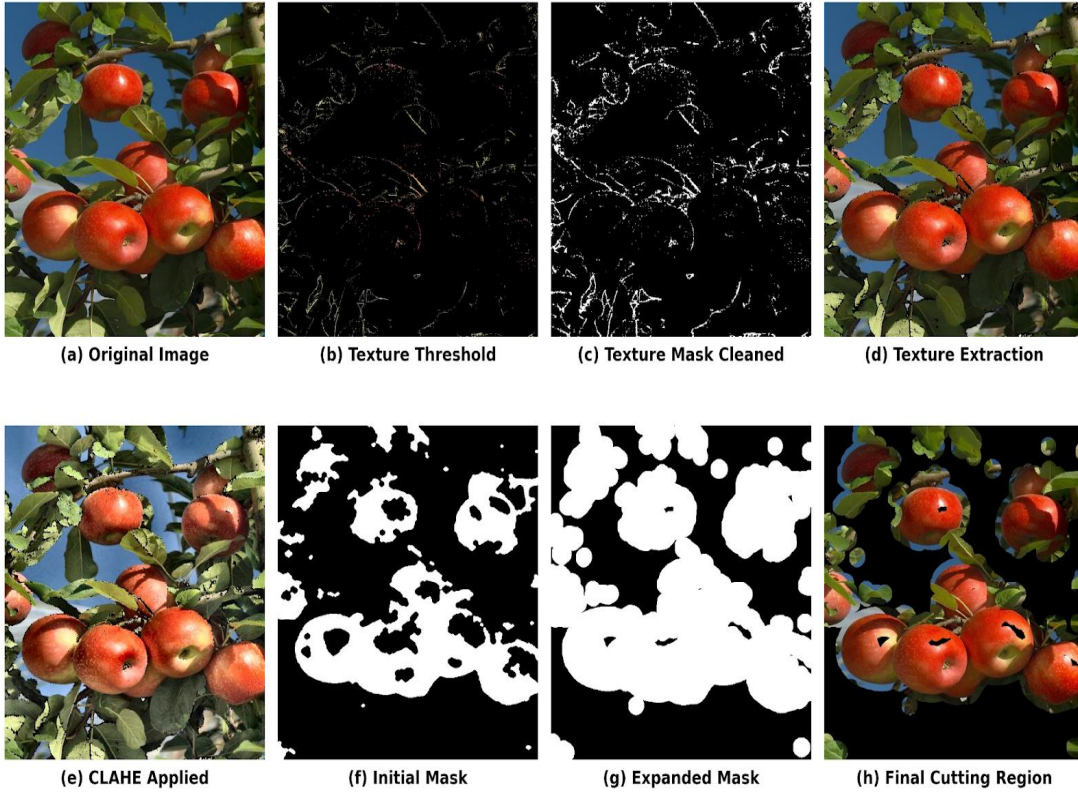


Fig. 2. Visualization of the preprocessing pipeline.

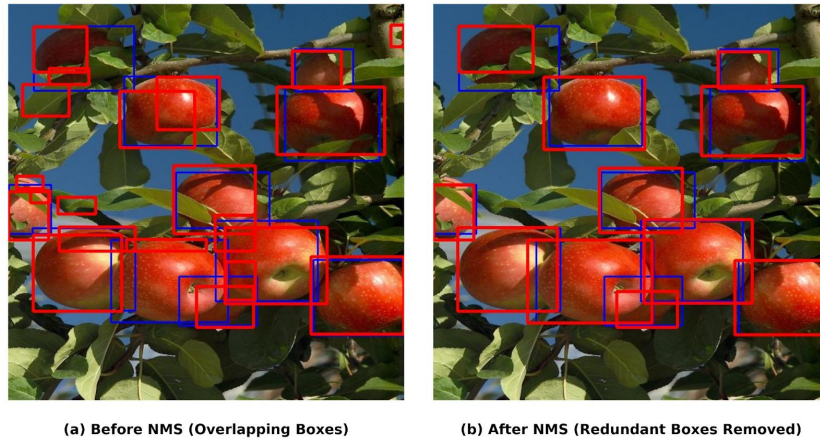


Fig. 3. Comparison of detection results before and after applying NMS.

3.2. Performance Analysis of Version 5

The final Version 5 model was evaluated on the Close and Far test sets. The evaluation metrics are defined as follows:

Metric Definitions:

F1-Score: The harmonic mean of Precision and Recall, used here as a reference for localization stability.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1)$$

Counting Metrics (Primary): Since the ultimate goal is yield estimation, counting accuracy was prioritized over bounding box precision.

- **MAE (Mean Absolute Error):** The average absolute difference between the predicted count (P) and the ground truth count (G) per image.

$$MAE = \frac{1}{N} \sum_{i=1}^N |P_i - G_i| \quad (2)$$

- **Counting Accuracy:** The average accuracy of the count for each image.

$$Accuracy = \frac{1}{N} \sum_{i=1}^N \left(1 - \frac{|P_i - G_i|}{G_i} \right) \times 100\% \quad (3)$$

Analysis Results:

- **Close-Range:** The model demonstrated high reliability with an F1-Score of 0.7094 and Accuracy of 81.36%. Notably, the MAE was exceptionally low (3.00), predicting 100 apples against a ground truth of 103.
- **Far-Range:** Despite high object density, the model maintained an F1-Score of 0.7019. The grid tiling approach proved essential; compared to processing raw images, Tiling improved Accuracy by 18.4% (to 78.32%) and significantly reduced MAE from 58.5 to 35.75.
- **Limitations:** Minor false negatives occurred where aggressive masking removed apple pixels, or when leaf colors closely resembled fruit.

3.3. Evolution of Models

Models 1 through 5 were evaluated to derive the optimal performance. The characteristics are summarized below.

TABLE I MODEL PERFORMANCE COMPARISON

Model	Close-Range						Far-Range					
	F1-Score	Precision	Recall	Accuracy	MAE	Pred /GT	F1-Score	Precision	Recall	Accuracy	MAE	Pred /GT
Version 1	-	-	-	-	-	-	0.6235	77.37%	52.22%	61.52%	45.5	274 / 406
Version 2	0.4242	96.55%	27.18%	37.06%	10.57	29 / 103	0.3197	76.64%	20.20%	43.62%	74.75	107 / 406
Version 3	0.4854	48.54%	48.54%	66.90%	4	103 / 103	0.0629	15.53%	3.94%	40.16%	75.75	103 / 406
Version 4	0.5909	71.23%	50.49%	70.27%	5.43	73 / 103	0.7013	72.14%	68.23%	73.23%	23.5	384 / 406
Version 5	0.7094	72.00%	69.90%	81.36%	3	100 / 103	0.7019	86.91%	58.87%	78.32%	35.75	275 / 406

- **Version 1 (Far):** As an initial baseline, it achieved an F1-Score of 0.6235 but suffered from a high MAE of 45.50, indicating insufficient precision for counting.
- **Version 2:** While Precision was high, Recall remained very low (20–27%), showing a strong tendency for under-fitting (missing most apples). The MAE spiked to 74.75, making it unsuitable for practical use.
- **Version 3:** It performed reasonably well on Close images (MAE 4.00) but failed on Far images (F1-Score 0.0629), revealing a lack of scale invariance.
- **Version 4:** The predecessor to Version 5, showed strong performance on Far images (F1 0.7013, MAE 23.50). The pipeline was fine-tuned based on this model to finalize Version 5.
- **Version 5 (Final Model):** As the optimized final iteration, this model

demonstrated robust performance across both scales. In the close-range scenario, it achieved an F1-Score of 0.7094 with an exceptionally low MAE of 3.00, indicating precise counting. In the challenging far-range scenario, it maintained a high F1-Score of 0.7019 and achieved a Counting Accuracy of 78.32%, proving that the hybrid pipeline effectively balances localization accuracy with counting precision even in complex environments.

References

- [1] A. Gongal, S. Amatya, M. Karkee, Q. Zhang, and K. Lewis, “Sensors and systems for fruit detection and localization: A review,” *Computers and Electronics in Agriculture*, vol. 116, pp. 8–19, Aug. 2015, doi: 10.1016/j.compag.2015.05.021.
- [2] I. Sa, Z. Ge, F. Dayoub, B. Upcroft, T. Perez, and C. McCool, “DeepFruits: A Fruit Detection System Using Deep Neural Networks,” *Sensors*, vol. 16, no. 8, p. 1222, Aug. 2016, doi: 10.3390/s16081222.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, June 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91.
- [4] L. Liu et al., “Deep Learning for Generic Object Detection: A Survey,” *International Journal of Computer Vision*, vol. 128, no. 2, pp. 261–318, Feb. 2020, doi: 10.1007/s11263-019-01247-4.
- [5] K. He, G. Gkioxari, P. Dollar, and R. Girshick, “Mask R-CNN,” in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2961–2969.
- [6] F. O. Unel, B. O. Ozkalayci, and C. Cigla, “The Power of Tiling for Small Object Detection,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Long Beach, CA, USA: IEEE, June 2019, pp. 582–591. doi: 10.1109/CVPRW.2019.00084.
- [7] N. Häni, P. Roy, and V. Isler, “MinneApple: a benchmark dataset for apple detection and segmentation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 852 – 858, 2020.
- [8] L61l, “Apple Computer Vision Model” *Roboflow Universe*, 2025. [Online]. Available: <https://universe.roboflow.com/a1-pi00p/apple-desti-ct8rq>.